

Architecture Diagram

Project SVARA — Sistem Pemesanan Tiket Konser Online



Gambar 1. Architecture Diagram alur Browser – Controller – Service – Repository – Database

Penjelasan Alur

Pada project saya, alur request dimulai dari Browser yang mengirim data lewat fetch API, diteruskan ke Controller (Svara2Controller.java), lalu ke Service (UserService, AdminService, PembayaranService), kemudian ke Repository, hingga sampai ke Database PostgreSQL yang saya hosting di Supabase. Frontend saya berupa halaman statis (HTML + Tailwind CSS + JavaScript) yang saya sajikan langsung oleh Spring Boot dari folder static, sehingga frontend dan backend saya jalankan dalam satu aplikasi yang sama, tidak terpisah host/port. Seluruh komunikasi frontend-backend saya lakukan menggunakan fetch API dengan format JSON.

Tabel Layer

Layer	Kelas / Interface	Peran
Controller	Svara2Controller	Menerima HTTP request (@RestController), meneruskan ke Service, dan mengembalikan ResponseEntity/JSON ke Browser. Menangani seluruh endpoint Admin, User, dan Pembayaran dalam satu kelas.
Service	UserService, AdminService, PembayaranService	Berisi logika bisnis: registrasi, autentikasi (pencocokan email & password), pembaruan data user, serta pemberian stempel waktu otomatis pada transaksi pembayaran sebelum disimpan.
Repository	UserRepository, AdminRepository, PembayaranRepository	Interface turunan JpaRepository<T, Integer> dari Spring Data JPA yang menyediakan operasi CRUD dan query turunan (mis. findByEmailAndPassword) ke database.
Entity	User, Admin, Pembayaran	Merepresentasikan struktur tabel database (@Entity) beserta atribut dan primary key auto increment (GenerationType.IDENTITY).
DTO	—	Pada project saya belum ada DTO layer terpisah; Controller menerima dan mengembalikan objek Entity secara langsung sebagai request/response body.

Alasan Pemilihan Arsitektur

Saya memilih arsitektur ini karena beberapa alasan. Pertama, arsitektur layered Controller-Service-Repository merupakan konvensi standar Spring Boot yang memisahkan tanggung jawab secara jelas, sehingga memudahkan saya dan tim (tiga orang) membagi pekerjaan dan memelihara kode. Kedua, dengan Spring Data JPA saya dapat melakukan operasi CRUD dan sebagian besar query cukup dengan mendefinisikan method signature, tanpa menulis SQL manual,

sehingga pengembangan menjadi lebih cepat. Ketiga, saya menyatukan backend dan frontend dalam satu aplikasi Spring Boot agar proses deployment ke Railway lebih sederhana, karena saya hanya perlu menjalankan satu service.